Showing MAWE to Dr. Hezy — a runbook

Everything you need in the order you need it. Follow it top to bottom.

---

0. Is the project ready to show?

Yes — with one gap you should state yourself before he finds it.

The engine	works end to end
The benchmark	built and validated
The comparison	built and validated
Tests	the full suite passes, on three Python versions
All four milestones	met
The one gap	**no run against a real AI model**

The gap is real and you should name it in your first two minutes. It is not a flaw in the work; it is the boundary of what the work currently shows, and the project is explicitly honest about it in the report, the deck and the poster.

The best way to say it:

> "Everything is built and tested, and two of our five hypotheses are answered by measurement. The other three compare AI models, and we haven't run that comparison yet because it needs a paid API key. The machinery to run it is finished — that's about an hour of compute once we have the key."

Say it early. A limitation you raise yourself reads as rigour; the same limitation found by your examiner reads as an oversight.

---

1. Five minutes before he arrives

Open a terminal in the project folder and run this once, so nothing is downloading or compiling while he watches:

```
pytest -q
```

It should end with a line saying everything passed. If it does, you are ready.

Have these open in tabs:

1. the terminal, in the project folder

2. `docs/reports/FINAL-REPORT.pdf`
3. the GitHub repository, on the Actions tab

---

2. The demo, in order

Six commands. You do not need all six — **A and B alone make the case**, and E is the one to run if he asks how any of this was tested without an API key.

A. The tests — start here

```
pytest -q
```

Say: "The whole project is covered by automated tests, and they run on Python 3.11, 3.12 and 3.13 in continuous integration on every push. No API key, no network, no cost."

Why this first: it is the single strongest thing you can show, and it takes about a minute.

B. Watch the agents work

```
python -m mawe.cli "Build a terminal snake game" --verbose
```

Say: "The Product Manager writes a specification with acceptance checks. The Developer implements it. The Reviewer actually *runs* the code in a sandbox and compares the output to the spec — it does not just read the code and give an opinion. If it rejects, the work goes back to the Developer."

If he asks whether that's a real AI model: "No — this is our mock provider, which replays scripted answers. It's deliberate: it makes the run deterministic and free, and it's the only way to test things a real model can't be made to do on demand, like returning byte-identical code twice."

C. The loop always terminates — the computer-science contribution

```
python scripts/week6_loop_stress.py
```

Say: "Two agents arguing is a loop, and a loop that doesn't terminate burns your entire API budget overnight. We drive a hundred thousand randomised adversarial review sequences through the routing policy. None fail to terminate and none exceed their iteration cap."

This is the result to lead with if he asks what is *novel*.

D. The benchmark is a valid instrument

```
python scripts/week9_bench_check.py
```

Takes about two minutes. Add `--quick` to skip the slow part.

Say: "Twenty tasks with hidden tests the agents never see, so scoring doesn't depend on our reviewing model — a run our own Reviewer approved can still score zero. And we measure the benchmark itself:

every task is solvable, no task can be passed by a program that does nothing, and when we inject deliberate bugs into the reference solutions the hidden tests catch 91% of them."

Why this matters: most student projects assert their benchmark is good. This one measures it.

E. How any of this is tested with no API key

Keep this one in your back pocket. Run it the moment he asks.

```
python scripts/how_we_test.py
```

It demonstrates, rather than claims: the single-method interface the engine talks to, a real program executed in a subprocess with an off-by-one error caught by running it, a termination guarantee that *only* a scripted provider can test, and a closing section stating what none of it proves.

F. The full comparison

```
python scripts/week11_experiment.py --tasks fizzbuzz palindrome json-query
```

Say — and do not skip this sentence: "These numbers come from a fixture with known answers, not from a real model. This shows the experiment pipeline works end to end and that the comparison can detect a quality difference. It says nothing about any model's ability."

The script prints that warning itself, so you are not relying on memory.

---

3. What to hand him

File	What it is
`docs/reports/FINAL-REPORT.pdf`	the full written report
`docs/reports/ABSTRACT.md`	the conference abstract, three lengths
`docs/poster/MAWE-URC-Poster-with-results.pdf`	the URC poster
the GitHub repository link	the code, the history and the CI record

---

4. Questions he is likely to ask

"Did you run this against GPT or Claude?"

> "Not yet. That's the one thing left. The adapters are written but have never run against a live API, so I'd call them untested rather than working. The benchmark, the baselines and the statistics are all finished and validated — it's one sweep of 240 runs once we have a key."

"So what have you actually proven?"

> "Two things, both by measurement rather than argument. The loop always terminates — a hundred thousand adversarial sequences, none failed. And compressing the revision history cuts the prompt by 94% at 80 iterations without changing the outcome. Those two don't need a model. The three that compare correctness and cost do."

"How did you test it worked, and how is that possible with no API key?"

The question most likely to decide how this goes. Answer with the distinction, then run the script.

> "Testing the machinery around a model and testing the quality of a model are two different jobs. The first needs no API key. The second does, and we haven't done it.
>
> Every agent talks to one interface with a single method. A real provider implements it by calling Anthropic; our mock implements it by replaying a scripted answer. The engine has no test mode and never imports the mock, so it genuinely cannot tell the difference — swapping to a real model is one environment variable.
>
> And the generated code is real. It's written to disk and executed in a subprocess, and we compare the actual bytes of its output against what the specification demanded. That part involves no API at all."

Then show him:

```
python scripts/how_we_test.py
```

Twenty seconds, four parts: the interface, a real program being run with an off-by-one error caught by execution, a guarantee only a scripted provider can test, and a plain statement of what none of it proves.

"Isn't using a mock avoiding the hard part?"

> "It's the opposite — some things can *only* be tested this way. Proving the no-progress exit fires needs the Developer to return byte-identical code twice. No real model will do that reliably; they're non-deterministic by design. Same for the reviewer that never approves, and code that cycles A to B and back. Those are the adversarial cases a safety guarantee depends on, and a real model won't produce them on demand."

"How do you know your benchmark is any good?"

> "We measured it. Every task has a reference solution that passes all its hidden tests. A program that starts and does nothing solves zero of twenty. And we inject single defects into the reference solutions — the tests catch 91% of them. That last one found two real bugs in our own test suite."

"What's the weakest part?"

> "Two things. No live model run, which I've mentioned. And our comparison between the multi-agent engine and the self-refining baseline changes two variables at once — role separation and the sandbox-grounded review — so a difference couldn't be attributed to one

> of them. We report that rather than hide it; separating them needs a fifth condition."

Answering that last one well is worth more than any demo. It shows you understand what your own experiment can and cannot support.

"Is the sandbox secure?"

> "No, and we say so. It's a subprocess with a timeout, path containment and output truncation — a resource limit, not a security boundary. Docker with no network was the stated hardening target and we didn't implement it."

---

5. If something breaks

Problem	Fix
`pytest` not found	`pip install -r requirements.txt` then `pip install -e .`
A JavaScript task is skipped	Node isn't installed. Harmless — the 12 Python tasks still run
A script seems to hang	The benchmark runs real subprocesses; week 9 takes ~2 min. Use `--quick`
Something fails on the day	Show `docs/reports/FINAL-REPORT.pdf` and the green CI history instead. The measurements are recorded there and the Actions tab proves they ran

---

6. The one-sentence version

If you only get thirty seconds:

> "We built a system where three AI agents — a planner, a developer and a reviewer that actually runs the code — work together instead of one model doing everything in one pass. We proved the loop always terminates and that our memory compression cuts prompt size by 94% without changing results. What's left is running it against a real model to see whether the extra structure is worth the cost."